

Pengantar Konsep Pemrograman Berorientasi Objek

1. Kompetensi

Setelah menempuh materi percobaan ini, mahasiswa mampu mengenal:

1. Perbedaan paradigma berorientasi objek dengan paradigma struktural
2. Konsep dasar PBO

2. Pendahuluan

2.1 Pemrograman prosedural vs berorientasi objek

Perbedaan mendasar antara pemrograman prosedural dengan pemrograman berorientasi objek (PBO) adalah: Pada pemrograman prosedural, program dipecah ke dalam sub-program atau **fungsi**. Sedangkan pada PBO, program dipecah ke dalam **object**, dimana objek tersebut membungkus **state** dan **method**.

Kelebihan PBO adalah program dapat lebih fleksibel dan modular, jika ada perubahan fitur, maka dapat dipastikan keseluruhan program tidak akan terganggu. Berbeda dengan prosedural, perubahan sedikit fitur saja kemungkinan dapat mengganggu keseluruhan program.

Untuk lebih jelas, berikut perbedaan antara pemrograman prosedural dengan pemrograman berorientasi object.

- **Procedural Programming**

J Procedural.java > ...

```
1 public class Procedural {
2     Run | Debug
3     public static void main(String[] args) {
4         String itemName1 = "Coffee";
5         float price1 = 120000.0f;
6         int quantity1 = 3;
7
8         String itemName2 = "Sugar";
9         float price2 = 30000.0f;
10        int quantity2 = 1;
11
12        System.out.println(itemName1 + ": " + total(price1, quantity1));
13        System.out.println(itemName2 + ": " + total(price2, quantity2));
14    }
15
16    static float total(float price, int quantity) {
17        return price * quantity;
18    }
19 }
```

- **Object Oriented Programming**

```
J OOP.java > ...
1  class Product {
2
3      private final String itemName;
4      private final float price;
5
6      public Product (String itemName, float price) {
7          this.itemName = itemName;
8          this.price = price;
9      }
10
11     public float total(int quantity) {
12         return price * quantity;
13     }
14
15     public String getName() {
16         return itemName;
17     }
18 }
19
20 public class OOP {
21     Run | Debug
22     public static void main(String[] args) {
23
24         Product product1 = new Product("Coffee", 120000.0f);
25         Product product2 = new Product("Sugar", 30000.0f);
26
27         System.out.println(product1.getName() + ": " + product1.total(3));
28         System.out.println(product2.getName() + ": " + product2.total(1));
29     }
30 }
```

Berdasarkan dua buah potongan kode tersebut, kode program dalam bentuk object lebih tertata dan lebih efisien karena tidak menggunakan kode yang berulang daripada kode prosedural. Hal tersebut dapat dilihat pada contoh di kode program menggunakan object, jelas bahwa object baru dibuat menggunakan kata kunci **new** yang diikuti oleh nama kelasnya.

NB:

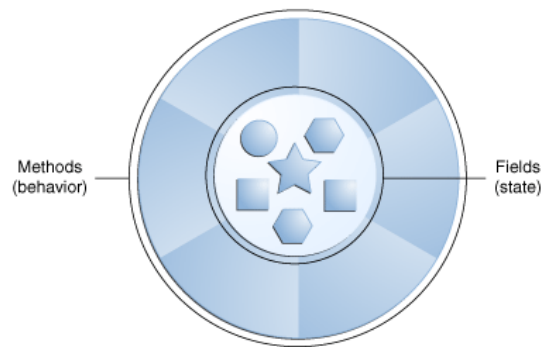
Di Dalam modul praktikum ini, kita akan mencoba membuat class, membuat object dan memanggil method-method yang ada di dalam object. Penjelasan mengenai anatomi class (atribut, method) akan dibahas lebih detail di modul praktikum berikutnya.

2.2 Konsep dasar PBO

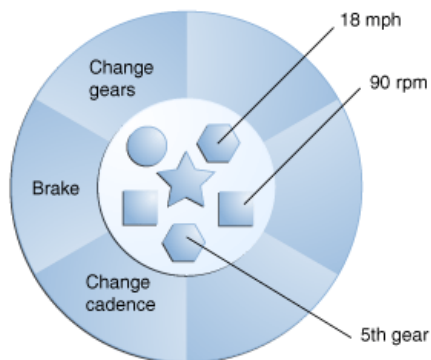
Di Dalam PBO dikenal beberapa aspek, yaitu:

a. Object

Object adalah suatu rangkaian dalam program yang terdiri dari **state** dan **behaviour**. Object pada software dimodelkan sedemikian rupa sehingga mirip dengan objek yang ada di dunia nyata. Objek memiliki state dan behaviour. State adalah ciri-ciri atau atribut dari object tersebut. Misal object Sepeda, memiliki state **brand**, **speed**, **gear** dan sebagainya. Sedangkan behaviour adalah perilaku yang dapat dilakukan object tersebut. Misal pada Sepeda, behaviour nya antara lain, **speed changes**, **gear changes**, **brake** dan lain sebagainya.



Gambar 1. Object pada software



Gambar 2. Sepeda yang dimodelkan sebagai object pada software

b. Class

Class adalah blueprint atau prototype dari object. Ambil contoh object sepeda. Terdapat berbagai macam sepeda di dunia, dari berbagai merk dan model. Namun semua sepeda dibangun berdasarkan blueprint yang sama, sehingga setiap sepeda memiliki komponen dan karakteristik yang sama. Sepeda yang anda miliki di rumah, adalah hasil **instansiasi** dari **class** sepeda.

c. Enkapsulasi

Disebut juga dengan **information-hiding**. Dalam berinteraksi dengan objek, seringkali kita tidak perlu mengetahui kompleksitas yang ada didalamnya. Contoh pada sepeda, ketika kita mengganti gear pada sepeda, kita tinggal menekan tuas gear yang ada di grip stang sepeda saja. Kita tidak perlu mengetahui bagaimana cara gear berpindah secara teknis.

d. Inheritance

Disebut juga **pewarisan**. Inheritance memungkinkan kita untuk mengorganisir struktur program dengan natural. Inheritance juga memungkinkan kita untuk memperluas fungsionalitas program tanpa harus mengubah banyak bagian program. Contoh di dunia nyata, objek sepeda dapat diturunkan lagi ke model yang lebih luas, misal sepeda gunung (mountain bike) dan road bike. Dimana masing-masing dapat memiliki komponen tambahan, misal sepeda gunung memiliki suspensi, yang tidak dimiliki sepeda biasa. Dalam hal ini, objek mountain bike dan road bike **mewarisi** objek sepeda.

e. Polimorfisme

Polimorfisme juga meniru sifat objek di dunia nyata, dimana sebuah object dapat memiliki bentuk, atau menjelma menjadi bentuk-bentuk lain. Misalkan saja objek pesawat terbang. Objek ini dapat diwariskan menjadi pesawat jet dan pesawat baling-baling. Keduanya memiliki kemampuan untuk menambah kecepatan. Namun secara teknis, metode penambahan kecepatan

antara pesawat jet dengan baling-baling tentu berbeda, karena masing-masing memiliki jenis mesin yang berbeda.

3. Percobaan

3.1 Percobaan 1

Di Dalam percobaan ini, kita akan mendemonstrasikan bagaimana membuat class, membuat object, kemudian mengakses method di dalam class tersebut.

1. Buka Netbeans atau menggunakan editor java, buat project **BikeDemo**.
2. Buat class Bike. Klik kanan pada package **BikeDemo** – New – Java Class.
3. Ketikkan kode class Bike dibawah ini.

```
1  public class Bike{
2      private String brand;
3      private int speed;
4      private int gear=1;
5      // Gear 1: max 5 km/h, Gear 2: max 10 km/h, ... Gear 6: max 60 km/h
6      private final int[] GEAR_SPEED_LIMITS = {5, 10, 25, 30, 40, 60};
7      public void setBrand(String brandName){
8          brand = brandName;
9      }
10     public void gearChanges(int gearValue){
11         if (gearValue < 1 || gearValue > 6) {
12             System.out.println("Invalid gear value. Gear must be between 1 and 6.");
13         }
14         else{
15             gear = gearValue;
16         }
17     }
18     public int speedAcceleration(int increment) {
19         speed += increment;
20         if (speed > GEAR_SPEED_LIMITS[gear - 1]) {
21             speed = GEAR_SPEED_LIMITS[gear - 1];
22         }
23         return speed;
24     }
25     public int speedDeceleration(int decrement) {
26         speed -= decrement;
27         if (speed < 0) {
28             speed = 0;
29         }
30         return speed;
31     }
32     public void printInfo(){
33         System.out.println("Brand : " + brand);
34         System.out.println("Speed : " + speed);
35         System.out.println("Gear : " + gear);
36     }
37 }
```

4. Kemudian pada class main, ketikkan kode berikut ini.

```

1  public class BikeDemo{
    Run | Debug
2      public static void main(String[] args){
3          Bike mountainBike1 = new Bike();
4          Bike mountainBike2 = new Bike();
5
6          mountainBike1.setBrand("Trek");
7          mountainBike1.speedAcceleration(10);
8          mountainBike1.gearChanges(2);
9          mountainBike1.printInfo();
10
11         mountainBike2.setBrand("Giant");
12         mountainBike2.speedAcceleration(20);
13         mountainBike2.gearChanges(3);
14         mountainBike2.printInfo();
15     }
16 }

```

5. Cocokkan hasilnya:

```

Brand : Trek
Speed : 5
Gear : 2
Brand : Giant
Speed : 5
Gear : 3

```

3.2 Percobaan 2

Di Dalam percobaan ini, akan didemonstrasikan salah satu fitur yang paling penting dalam PBO, yaitu inheritance. Disini kita akan membuat class **RoadBike** untuk kelas sepeda balap yang mana adalah turunan/warisan dari class Bike. Pada dasarnya class RoadBike adalah sama dengan class Bike, hanya saja pada road bike terdapat **lebar ban**. Untuk itu kita tidak perlu membuat class Sepeda Gunung dari nol, tapi kita wariskan saja class Sepeda ke class SepedaGunung.

Penjelasan lebih detail tentang inheritance akan dibahas pada modul selanjutnya.

1. Masih pada project **BikeDemo**. Buat class **RoadBike**.
2. Tambahkan kode **extends Bike** pada deklarasi class RoadBike. Kode extends ini menandakan bahwa class RoadBike mewarisi class Bike.

```

1  public class RoadBike extends Bike {
2
3  }

```

3. Kemudian lengkapi kode RoadBike seperti berikut ini:

```

1 public class RoadBike extends Bike {
2     private int tireWidth;
3
4     public void setTireWidth(int width) {
5         tireWidth = width;
6     }
7
8     @Override
9     public void printInfo() {
10        super.printInfo();
11        System.out.println("Tire Width : " + tireWidth + " mm");
12        System.out.println("Bike Type : Road Bike");
13    }
14 }

```

4. Kemudian pada class main, tambahkan kode berikut ini:

```

1 public class BikeDemo{
2     Run | Debug
3     public static void main(String[] args){
4         Bike mountainBike1 = new Bike();
5         Bike mountainBike2 = new Bike();
6         RoadBike roadBike1 = new RoadBike();
7
8         mountainBike1.setBrand("Trek");
9         mountainBike1.speedAcceleration(10);
10        mountainBike1.gearChanges(2);
11        mountainBike1.printInfo();
12
13        mountainBike2.setBrand("Giant");
14        mountainBike2.speedAcceleration(20);
15        mountainBike2.gearChanges(3);
16        mountainBike2.printInfo();
17
18        roadBike1.setBrand("Specialized");
19        roadBike1.setTireWidth(25);
20        roadBike1.speedAcceleration(15);
21        roadBike1.gearChanges(4);
22        roadBike1.printInfo();
23    }
24 }

```

5. Cocokkan hasilnya:

```
Brand : Trek  
Speed : 5  
Gear : 2  
Brand : Giant  
Speed : 5  
Gear : 3  
Brand : Specialized  
Speed : 5  
Gear : 4  
Tire Width : 25 mm  
Bike Type : Road Bike
```

4. Kesimpulan

Dari percobaan di atas, kita telah mendemonstrasikan bagaimana paradigma pemrograman berorientasi objek dan mengimplementasikannya kedalam program sederhana. Kita juga telah mendemonstrasikan salah satu fitur paling penting dari PBO yaitu **inheritance**, yaitu dalam hal membuat class **RoadBike**.

Kita ketahui bahwa RoadBike pada dasarnya adalah sama dengan **Bike** (memiliki gear, memiliki kecepatan, dapat menambah kecepatan, dapat mengerem, pindah gigi) namun ada fitur tambahan yaitu **lebar ban**. Maka kita tidak perlu membuat class RoadBike dari nol, kita **extends** atau wariskan saja dari class Bike, kemudian kita tinggal tambahkan fitur yang sebelumnya belum ada di class Bike. Inilah salah satu kelebihan PBO yang tidak ada di pemrograman prosedural.

5. Pertanyaan

1. Jelaskan perbedaan antara object dengan class!
2. Jelaskan alasan gear dan brand dapat menjadi atribut dari object Bike!
3. Sebutkan salah satu kelebihan utama dari pemrograman berorientasi objek dibandingkan dengan pemrograman prosedural!
4. Apakah diperbolehkan melakukan pendefinisian dua buah atribut dalam satu baris kode seperti **"public String nama, alamat;"**?
5. Pada class RoadBike, jelaskan alasan atribut brand, speed, dan gear tidak lagi ditulis di dalam class tersebut!

6. Tugas Praktikum

1. Lakukan langkah-langkah berikut supaya tugas praktikum yang dikerjakan tersistematis:
 - a. Foto 4 buah objek di sekitar kalian dengan 2 objek di antaranya merupakan objek yang mengandung konsep pewarisan (inheritance), contoh: kulkas, kursi, meja ruang tamu, meja belajar sehingga diketahui meja ruang tamu dan meja belajar mewarisi objek meja!
 - b. Lakukan pengamatan terhadap 4 objek tersebut untuk menentukan atribut dan methodnya!
 - c. Berdasarkan 4 buah objek tersebut, buat class nya dalam Bahasa pemrograman Java!
 - d. Perlu diperhatikan bahwa terdapat dua class hasil pewarisan sehingga perlu menambah satu class baru sebagai class yang mewarisi dua class tersebut!
 - e. Tambahkan dua atribut untuk setiap class!
 - f. Tambahkan tiga method untuk setiap class termasuk method cetak informasi!
 - g. Tambahkan satu class Demo sebagai main!

- h. Instansiasikan satu buah objek untuk setiap class!
- i. Terapkan setiap method untuk setiap objek yang dibuat!
- j. Contoh yang telah disebutkan pada poin 1.a tidak diperbolehkan dipakai dalam pengerjaan tugas praktikum ini!